HPCA Chapter 2

15 March 2025 11:23

Vid 25 : We will learn about Latency and throughput which measure a computer's performance. Then in further topics we will look at benchmarks and few general methods to improve performance.

Vid 26 : Throughput is number of operations that can be done in a second . Whereas Latency is the time taken for something to be completed from start to done.

Vid 27 & 28 : Quiz to illustrate Latency not always equal to 1/Throughput , Throughput is increased with replication of processing units (or other techniques).

Vid 29 : Comparing performance, there are two processors x and y,
Speedup N = speed(x)/speed(y)
= Throughput(x)/ Throughput(y)
= Latency(y)/ Latency(x)

Vid 30, 31, 32, 33 and 34: Speed UP Quiz, Intuitively, a speed up of Greater than 1 is actually speedup(Lower latency or higher throughput) if its less than one new thing is slower than the old.

Vid 35 : Measuring performance :
We know performance is 1/ Execution time , but execution time on what workload??
We may choose to run actual user data, but what all programs we need to run ? how do we get that data? What's guarantee that all users use the machine to perform same computations? .
So instead of actual workloads, we use benchmark workloads.

Vid 36 : Benchmarks : They are the workloads (programs and input data) that users and companies have agreed upon to measure performance. Usually we don't have just one benchmark program we have a benchmark suite. They contain multiple programs (each representative of some type of apps) along with their input data.

Vid 37 : Types of benchmarks :

1. Real Applications : they are most representative, but also most difficult to setup(like we are building a new computer, it would be difficult to setup a full-fledged OS on it, and have all the graphics card /hard drives and other setup. So we can't run a web browser or a database on this setup. They are used on machines that are available in market for consumer usage.
2. Kernels : we are going to find the most time consuming part of an application, like some sort of an loop and we are going to use this for testing performance of machines. Sometimes if we are in early stages of development we may not have compiler setup for it so running kernel is still difficult. Once a machine is built , in the form of **prototype** we can use kernels to measure its performance.
3. Synthetic benchmarks : they are similar to kernels but are simpler to compile. These are usually good for **design studies** of potentially new machines. But not so good for actually reporting performance to others.
4. Peak performance : instead of judging machines based on some programs they are running we can compare machines based on their peak performance, in theory, how many instructions per second this machine can execute. This is usually used for **marketing** and doesn't really tell it's goodness for real applications.

Vid 38 & 39 : Benchmarks Quiz :
Which of the following benchmarks use the code from real applications but doesn't use all of the code.

a. Actual workload mix
b. Application kernel
c. Synthetic Benchmarks
d. Application suite
e. Peak performance
   Sol : (e) doesn't use any code, (c) code is not from real applications but specifically designed for testing, (b) uses code from real applications and its reduced to form a kernel, so its right (d) it uses code from real apps but uses all of the code, (a) is similar to (d) but it is actual user's usage pattern

Vid 40 : Benchmark standards. There are standards organizations that put together benchmark suites for comparing performance between machines. They take inputs from user groups experts in academia and from manufacturers to come up with benchmark suites along with instructions to produce representative measurements. Some of them are :
TPC : used for databases, webservers, data mining and data transaction processing
EEMBC : for embedded processing
SPEC : typically engineering workloads and for raw processors, mostly processor oriented rather than being io intensive.

Vid 41 : Summarizing performance
Given below is comparision table of two computers running three applications, speedup of x over y, when seen average execution time of all apps on comp x its 8 , on y its 12, so oerall average speedup of x over y is 1.5. One might think we may average the speedups, which would come to (2+0.7+2.2)/3 = 1.63, but this approach is wrong as we can't take avg of ratios here. Instesad if we really want to operate on speedups, we can take their Geometric mean, which is equal to ratio of GMs of Execution times as shown below.

$$\Rightarrow \quad \frac{\sqrt[3]{9 \cdot 10} \cdot 5}{\sqrt[3]{18 \cdot 7 \cdot 11}} = \sqrt[3]{\frac{9}{18} \cdot \frac{10}{7} \cdot \frac{5}{11}}$$

	Comp x	Com y	Speedup
App A	9	18	2
App B	10	7	0.7
App C	5	11	2.2
Average	8	12	1.5
Geo Mean	7.66	11.15	1.456

Vid 42 : Speed UP Quiz : We have a new laptop, its speed up on three tasks are as follows, on Homework formatting its 2, and on virus scan its 8. the overall speedup is:

Vid 43 : as we don’t have actual times of execution, to calculate average execution time and take ratio, all we can do is take geometric mean, so overall speedup is 4.

Vid 44 : Iron's law of performance
CPU time : Number of instructions in program * --> Affected by Algorithm, Compiler & Instruction set
Clocks per instruction * -->Instruction set and Processor Design
Clock period --> Processor Design, Circuit Design & Transistor Physics

we decompose CPU time into these three times as they allow us to think about different aspects of computer architecture and surrounding areas that we can change. A simpler instruction set might take

more of them for given algorithm , but clocks per instruction can be low, also clock cycle time can be low compared to complex instruction set.There are several techniques to spend fewer cycles per instruction without changing the instruction set. A processor design might do very small tasks per clock making the clock cycle low at an expense of possibly increase the number of clocks to execute that instruction/algo. so a good processor design will balance Clocks per instruction and Clock period, whereas a good ISA will balance Number of instructions per program and Clocks per instruction.

Vid 45 and 46: we have a cpu with 3 B instructions, 2 cycles each instruction, and 3Ghz , time for execution ? 2s

Vid 47 : CPU time = $\sum_i IC_i \times CP_i * T$; when there are different types of instructions, instruction count of type *i* , clocks per instruction for instructino of type *i,* times clock period.

Vid 48 & 49 : There are 50 B instructions, 10B are Branches (4CPI), 15 B are Loads(2 CPI), 5B are stores(3 CPI), rest are integer add with CPI 1. CPU running at 4 Ghz, what is CPU time.
sol : 10 + 7.5 + 3.75 + 5 = 26.25

Vid 50 :Amdhal's law, we use this when speed up is only some part of the program, only some instructions and so on.

$$\text{Speed up } = \frac{1}{(1-FRAC_{ENH})+\frac{FRAC_{ENH}}{SPEEDUP_{ENH}}}$$

FRAC $_{ENC}$ is the % of original execution time effected by enhancement.

Vid 51 & 52 : Amdahl's law quiz
program has 50B instructions with composition and respective CPI shown, clock freq. of 2 Ghz,if branch instruction's cpi is improved from 4 to 2, what is the overall speedup.

Instn. type	% of total number of instructions	CPI
int	40	1
branch	20	4
load	30	2
store	10	3

Execution time before speedup : (.4 + .2*4 + .3*2 + .1*3) * 50B/2Ghz
Execution time after speedup : (.4 + .2*2 + .3*2 + .1*3) * 50B/2Ghz
Speedup = Execution time $_{new}$/ Execution time $_{old}$ = 1.24
We can also solve this with the Amdhal's law equation, for that we need to figure out the time consumed by branches and plug it into equation.

Vid 53 : Amdhal's law's implications :
Its better to have small improvement on large part of execution times rather than big improvemets on things that are small part of execution times . For example, having 20x speedup on thing that just consumes your 10% of execution time is far worse than speedup of just 1.6 on thing that's 80% of execution time.

$$\frac{1}{0.9+\frac{0.1}{20}} = 1.105$$

$$\frac{1}{0.2+\frac{0.8}{1.6}} = 1.43$$

In fact, even an theoritical infinite speedup on 10% of execution time is just 11% or $1.111(=1/0.9)$ overall speedup. In other words, Make the common case fast.

Vid 54 : Another Amdhal's law Quiz

Say the processor speed is 2 Ghz, and we are considering these possible improvements:

1. increase the CPI of branch from 4 to 3
2. increase the clock from 2 to 2.3
3. reduce the store cpi from 3 to 2

Assuming we can do only one of these which one would you choose?

Instn. type	% of total execution time	CPI
int	40	1
branch	20	4
load	30	2
store	10	3

for the (1.) improvement is

$$\frac{1}{0.8 + \frac{0.2}{1.33}} = 1.05$$

for (2.) improvement is 2.3/2 or 1/(0+1/1.15) = 1.15

for (3.) improvement is 1/(0.9 + 0.1/1.5) = 1.0345

Option (2.) is the gives the good improvement over all options, this is because it affects 100% of the execution time.

Vid 55 : Lhadma's law:

its just amdhal spelled backwards, it says that while trying to improve the common case , don't messup the uncommon case too badly

for example : we achieved improvement of 2x on 90% of execution time but the rest 10% is slowed down by 10x.

then speed up = 1/(0.9/2 + 0.1/0.1) = 0.6897

if you have huge worsening on even a small part of the program, it might result in overall slowdown

Vid 56 : Law of Diminishing Returns

say in gen1: we have processor with ****_****_&&&&_&&&& execution time

in gen2 we have speedup of 2 on & type process so : ****_****_&&&& , overall 1/(0.5 + 0.5/2) = 1.3333 compared to gen1

in gen 3 we have speedup of 2 on & type process so : ****_****_&& , overall 1/(0.67 + 0.33/2) = 1.1976 compared to gen2

Initially its easy to get speedups, as we are done improving on that to get same level of speedup on same thing its very difficult, also not adding much value. therefore the solutions that improve the * part is much cheaper and gives good results than improving the & part. so NOT GO OVERBOARD IMPROVING THE SAME THING. things that have already improved are now a smaller percentage of execution time, hence less improvement, now we need to reaccess what is the dominant part in execution time

Vid 57: now we know how to measure and compare performance and we know overall performance improvement requires a careful balance of improving one aspect of the design and not messing up with the other. before we look into more advanced methods of improving performance and efficiency, we will look into Pipelining.